Un Player, due rappresentazioni geometriche

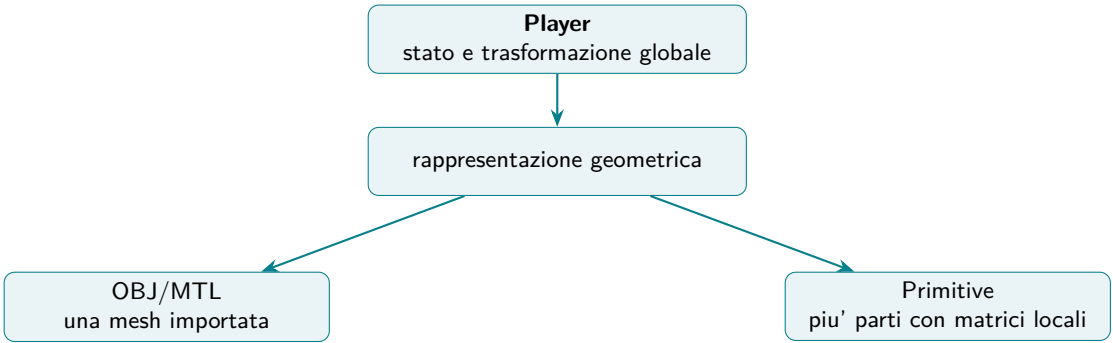
Assemblaggio di primitive e importazione OBJ/MTL
Geometrie, istanze e Model Matrix nel progetto Cannone 2D

Laboratorio di Computer Graphics – OpenGL

Obiettivi della lezione

- Comprendere la distinzione tra **logica del gioco** e **rappresentazione grafica**.
- Analizzare la classe `Player` realmente utilizzata nel progetto.
- Costruire il cannone con **cerchio e rettangolo** oppure importarlo da **OBJ/MTL**.
- Comprendere `MeshData`, `Scene::addGeometry()` e `Scene::addInstance()`.
- Applicare le trasformazioni con le **Model Matrix**, senza modificare i vertici.
- Capire come farfalle e cuori condividono geometrie e ricevono trasformazioni indipendenti.

L'idea fondamentale: separare geometria e comportamento



Principio progettuale

La logica di puntamento e sparo **non deve dipendere** dal modo in cui e' stata costruita la geometria del player.

Le strutture dati: MeshData e GeometryInstance

MeshData

- Vertici della geometria (posizioni).
- Colori dei vertici.
- Tipo di primitiva OpenGL.

```
MeshData mesh;
mesh.primitiveType = GL_TRIANGLES;
// mesh.positions, mesh.colors
```

GeometryInstance

- `geometryIndex`: quale mesh usa.
- `modelMatrix`: dove si trova.
- `visible`: se viene disegnata.

```
struct GeometryInstance {
 size_t geometryIndex = 0;
 glm::mat4 modelMatrix{1.0f};
 bool visible = true;
};
```

Geometria e istanza: due operazioni differenti

```
const std::size_t geometry = scene.addGeometry(mesh);
const std::size_t first = scene.addInstance(geometry);
const std::size_t second = scene.addInstance(geometry);
```

- `addGeometry(mesh)` registra una **geometria** (VAO/VBO gestiti da `Scene`).
- `addInstance(geometry)` crea una **istanza** che fa riferimento alla stessa geometria.
- Le istanze possono avere **Model Matrix** e visibilita' diverse.

Esempio

Due farfalle con gli stessi triangoli possono comparire in posizioni differenti senza duplicare la mesh.

La classe Player: interfaccia effettiva

```
class Player {
public:
 enum class GeometryMode { OBJ, Primitives };
 struct Part {
   MeshData mesh;
   glm::mat4 localTransform{1.0f};
 };
 explicit Player(std::string objFilename);
 explicit Player(std::vector<Part> parts);
 bool initialize(Scene& scene);
 void setModelMatrix(Scene& scene, const glm::mat4& model);
 // ...
};
```

- Il **costruttore** sceglie la rappresentazione.
- `initialize()` crea geometrie e istanze nella scena.
- `setModelMatrix()` aggiorna tutte le parti del player.

Player da OBJ: scelta del file nel costruttore

```
Player player("assets/myshape.obj");
const bool playerReady = player.initialize(scene);
if (!playerReady) {
    std::cerr << "Errore caricamento player\n";
    // Gestione dell'errore e rilascio delle risorse
}
```

- Il costruttore memorizza il percorso dell'OBJ.
- `initializeOBJ()` chiama `PlayerShapeLoader::load()`.
- Il loader legge la geometria e i colori del materiale MTL.
- I vertici **non vengono centrati o scalati** nel loader.

Nota

Il file MTL viene individuato tramite il riferimento `mtllib` nell'OBJ, se supportato dal loader.

Player da primitive: un cannone semplice

```
std::vector<Player::Part> playerParts;
const glm::vec4 barrelColor(0.23f, 0.67f, 0.95f, 1.0f);
const glm::vec4 baseColor(0.98f, 0.78f, 0.25f, 1.0f);

// Canna rettangolare: da x=0 a x=24
playerParts.push_back({
    makeLink(24.0f, 6.0f, barrelColor), glm::mat4(1.0f)
});

// Base circolare: centro nell'origine locale
playerParts.push_back({
    ShapeGenerator::createCircle(0,0,7,7,32,baseColor,baseColor),
    glm::mat4(1.0f)
});
```

Perche' funziona?

Le due parti sono definite in coordinate **locali** del player, non in coordinate mondo.

Le parti del Player e le trasformazioni locali

```
struct Part {
    MeshData mesh;
    glm::mat4 localTransform{1.0f};
};
```

$$\boxed{M_i^{\text{world}} = M_{\text{player}}\, M_i^{\text{local}}}$$

- M_{player}: movimento e orientamento **dell'intero player**.
- M_i^{local}: posizione e orientamento **della singola parte**.
- Per il cannone base e canna possono avere trasformazione locale identita'.
- Per un robot, braccia e testa possono avere trasformazioni locali differenti.

Che cosa fa initialize(scene)?

```
// Per ogni parte del player (schema del codice reale):
const auto geometry = scene.addGeometry(part.mesh);
const auto instance = scene.addInstance(geometry);
// Memorizza l'indice dell'istanza e la matrice locale.
```

- **OBJ**: una mesh importata, inizializzata come una parte.
- **Primitive**: una parte per ogni geometria fornita al costruttore.
- In entrambi i casi, `Player` memorizza gli indici delle istanze.

Attenzione

Il metodo `initialize()` non disegna direttamente: prepara i dati per il renderer.

La Model Matrix globale del player

$$\boxed{M_{\text{player}} = T_{\text{base}}\, R_{\text{aim}}\, M_{\text{local}}} \qquad M_{\text{local}} = T_{\text{local}} R_{\text{local}} S_{\text{local}}$$

```
const glm::mat4 Tbase = glm::translate(
    glm::mat4(1.0f), glm::vec3(BASE_POSITION, 0.0f));
const glm::mat4 Raim = glm::rotate(
    glm::mat4(1.0f), glm::radians(angleDeg),
    glm::vec3(0.0f, 0.0f, 1.0f));
const glm::mat4 playerModel = Tbase * Raim * localModel;
player.setModelMatrix(scene, playerModel);
```

Ordine

Con vettori colonna, le trasformazioni agiscono da **destra verso sinistra**.

Come costruire localModel: OBJ oppure primitive

```
glm::vec3 localTranslation;
glm::vec3 localScale;
float localRotationDegrees;

if (PLAYER_GEOMETRY_MODE == Player::GeometryMode::OBJ) {
    localTranslation = glm::vec3(12.0f, 0.0f, 0.0f);
    localScale = glm::vec3(14.2836f, 14.2836f, 1.0f);
    localRotationDegrees = -90.0f;
} else {
    localTranslation = glm::vec3(0.0f);
    localScale = glm::vec3(1.0f);
    localRotationDegrees = 0.0f;
}
```

- Valori OBJ: **esempio specifico**, da adattare al file importato.
- Primitive: identita' se costruite nel riferimento desiderato.

Un'unica composizione, per entrambe le modalita'

```
glm::mat4 localModel(1.0f);
localModel = glm::translate(localModel, localTranslation);
localModel = glm::rotate(localModel,
    glm::radians(localRotationDegrees),
    glm::vec3(0.0f, 0.0f, 1.0f));
localModel = glm::scale(localModel, localScale);

const glm::mat4 playerModel = Tbase * Raim * localModel;
player.setModelMatrix(scene, playerModel);
```

Vantaggio

Non serve distinguere OBJ e primitive quando si aggiorna la trasformazione globale: cambia soltanto `localModel`.

Come Player applica la matrice a tutte le parti

```
void setModelMatrix(Scene& scene, const glm::mat4& model) {
    m_model = model;
    for (std::size_t i = 0; i < m_instances.size(); ++i)
        scene.instance(m_instances[i]).modelMatrix =
            m_model * m_localTransforms[i];
}
```

- Una sola chiamata aggiorna il player completo.
- Ogni parte conserva la sua posizione **relativa**.
- Non vengono modificati i vertici nei VBO durante il movimento.

Il punto di sparo: un punto nel riferimento locale

Canna a primitive

$p^{\text{local}}_{\text{muzzle}} = (24, 0, 0, 1)^T$ se la canna termina a $x = 24$.

OBJ

$p^{\text{local}}_{\text{muzzle}}$ dipende dalla forma modellata; puo' essere $(0, 0, 0, 1)^T$.

$$\boxed{p^{\text{world}}_{\text{muzzle}} = M_{\text{player}}\, p^{\text{local}}_{\text{muzzle}}}$$

```
const glm::vec4 muzzleWorld4 =
    playerModel * glm::vec4(muzzleLocal, 1.0f);
const glm::vec2 muzzle(muzzleWorld4.x, muzzleWorld4.y);
```

Distinzione

La posizione iniziale del proiettile dipende dal punto di sparo; la direzione puo' dipendere separatamente dal mouse.

I target: una struttura logica e un'istanza grafica

```
enum class TargetType { Butterfly, Heart };
struct Target {
    std::size_t instanceIndex = 0;
    TargetType type = TargetType::Butterfly;
    glm::vec2 position{0.0f};
    glm::vec2 direction{1.0f, 0.0f};
    float directionTimer = 0.0f;
    bool alive = true;
};
```

- `Target`: posizione, direzione e stato **nella logica del gioco**.
- `instanceIndex`: collegamento all'istanza **nella scena**.
- Il target non deve necessariamente conoscere come e' stata costruita la mesh.

Target da primitive oppure da OBJ: cambia la sorgente

Geometria procedurale

```
MeshData butterflyMesh =
 ShapeGenerator::createButterfly(
  0, 0, 0.05f, 0.05f, 80,
  colorA, colorB);
```

Geometria importata

```
MeshData butterflyMesh;
std::string error;
if (!PlayerShapeLoader::load(
 "assets/shape.obj",
 butterflyMesh, error)) {
 // Gestire l'errore
}
```

```
// Da qui in poi il percorso e' identico:
const auto butterflyGeometry = scene.addGeometry(butterflyMesh);
```

Precisazione sul progetto allegato

Nel `main_studenti.cpp` fornito la farfalla e' creata con `ShapeGenerator::createButterfly()`. Il caricamento da OBJ qui e' la **variante da introdurre**, non una chiamata gia' presente in quel file.

Una geometria, molte istanze di target

```
std::vector<Target> targets;
targets.reserve(MAX_BUTTERFLIES + MAX_HEARTS);

for (int i = 0; i < MAX_BUTTERFLIES; ++i) {
    Target b;
    b.type = TargetType::Butterfly;
    b.instanceIndex = scene.addInstance(butterflyGeometry);
    targets.push_back(b);
    scene.instance(b.instanceIndex).visible = false;
}
```

- `reserve()` prenota memoria, **non crea** i target.
- `addInstance()` crea una nuova occorrenza della mesh condivisa.
- `visible=false` la nasconde fino all'attivazione.

Dove nasce la posizione dei target?

```
for (Target& t : targets) {
    respawnTarget(t, targets, scene);
    scene.instance(t.instanceIndex).visible = true;
}
```

- `respawnTarget()` genera una posizione candidata casuale.
- La verifica `freePosition(candidate)` evita sovrapposizioni con target attivi.
- Se la posizione e' accettata: `b.position = candidate;`.
- La funzione inizializza anche `direction` e `directionTimer`.

Attenzione

La posizione `b.position` e' uno stato logico: non modifica direttamente i vertici della geometria.

Dove viene costruita la Model Matrix dei target?

```
for (Target& b : targets) {
    float sc;
    if (b.type == TargetType::Butterfly)
        sc = BUTTERFLY_OBJ_SCALE;
    else
        sc = 8.0f;

    scene.instance(b.instanceIndex).modelMatrix =
        translate2D(b.position) * scale2D(sc, sc);
}
```

$$\boxed{M_b(t) = T(b.position(t))\, S(sc, sc)}$$

- La stessa formula funziona per OBJ e primitive.
- Il fattore sc va scelto in base alle dimensioni della mesh.

Animazione: aggiornare la posizione e poi la matrice

```
// Nel ciclo di aggiornamento dei bersagli:
b.position += b.direction * speed * dt;

instance.modelMatrix =
    translate2D(b.position) * scale2D(scale, scale);
```

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \mathbf{d}(t)\, v \Delta t$$

- Il movimento modifica lo stato `b.position`.
- La Model Matrix traduce quello stato in una trasformazione grafica.
- `dt` rende il movimento meno dipendente dal frame rate.

Percorso completo fino al rendering

